Laboratorio 8

2026-04-26

Contents

Preparacion del entorno analitico . . . 2
Carga y diagnostico inicial del conjunto de datos . . . 2
Reutilizacion del particionamiento experimental . . . 4
Diagnostico inicial de predictores . . . 7
Transformaciones para modelado SVM . . . 12
Cierre del avance . . . 14
Diseno experimental de los modelos SVM . . . 14
Evaluacion de kernels para clasificacion de precios . . . 16
- Kernel lineal . . . 16
- Kernel radial . . . 18
- Kernel polinomial . . . 19

Prediccion de categorias de precio . . . 20
Primer analisis comparativo de desempeno . . . 21
Evaluacion de la clasificacion en el conjunto de prueba . . . 23
Analisis de errores por categoria de precio . . . 25
Diagnostico de generalizacion de los modelos SVM . . . 27
Implicaciones del ajuste para la toma de decisiones de SmartStay Advisors . . . 28
Recomendaciones para mejorar el desempeno predictivo . . . 28
Contraste de desempeno entre configuraciones SVM . . . 29
Lectura operativa de los errores de clasificacion . . . 30
Implicaciones para la categorizacion de precios . . . 33
Seleccion del clasificador SVM mas estable . . . 34
Redaccion integrable al informe ejecutivo . . . 35
Preparacion del problema de prediccion de precios . . . 35
Transformacion de predictores para SVR . . . 35
Ajuste de modelos de regresion con vectores de soporte . . . 37
- SVR con kernel lineal . . . 37
- SVR con kernel radial . . . 37

SVR con kernel polinomial . . . 37
Evaluacion del error predictivo . . . 38
Diagnostico visual de predicciones . . . 38
Comparacion con modelos de regresion anteriores . . . 40
Comparacion con clasificadores desarrollados previamente . . . 41
Verificacion final de objetos . . . 42
Sintesis del desempeno de clasificacion . . . 42
Diagnostico global de ajuste . . . 43
Sintesis del desempeno de regresion . . . 44
Resumen ejecutivo de decisiones . . . 45
Recomendaciones finales para SmartStay Advisors . . . 46

Preparacion del entorno analitico

```
paquetes_requeridos <- c("tidyverse", "caret", "e1071", "kernlab", "knitr")
paquetes_opcionales <- c("MLmetrics")

for (pkg in paquetes_requeridos) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    stop(paste("Falta instalar el paquete requerido:", pkg))
  }
  suppressPackageStartupMessages(library(pkg, character.only = TRUE))
}

for (pkg in paquetes_opcionales) {
  if (requireNamespace(pkg, quietly = TRUE)) {
    suppressPackageStartupMessages(library(pkg, character.only = TRUE))
  } else {
    message("Paquete opcional no disponible en este entorno: ", pkg)
  }
}

set.seed(123)
```

Este documento prepara los datos para clasificacion con SVM sobre la variable objetivo categorica de precio. Se prioriza reproducibilidad, limpieza del flujo y trazabilidad del particionamiento experimental.

Carga y diagnostico inicial del conjunto de datos

```
objetos_cargados <- load("listings.RData")

if ("listings" %in% objetos_cargados) {
  datos <- get("listings")
} else {
  candidatos_df <- objetos_cargados[sapply(objetos_cargados, function(x) is.data.frame(get(x)))]
```

```r
  if (length(candidatos_df) == 0) {
    stop("No se encontro ningun data frame en listings.RData")
  }
  datos <- get(candidatos_df[1])
}

datos <- as.data.frame(datos)
```

```r
cat("Objeto usado:", if ("listings" %in% objetos_cargados) "listings" else "primer data.frame encontrado
```

```
## Objeto usado: listings
```

```r
cat("Dimensiones:", nrow(datos), "filas x", ncol(datos), "columnas\n")
```

```
## Dimensiones: 171748 filas x 80 columnas
```

```r
resumen_tipos <- tibble(
  variable = names(datos),
  clase = sapply(datos, function(x) paste(class(x), collapse = ", ")),
  n_unicos = sapply(datos, function(x) dplyr::n_distinct(x, na.rm = TRUE)),
  na_count = sapply(datos, function(x) sum(is.na(x))),
  na_pct = round(100 * na_count / nrow(datos), 2)
)

resumen_tipos %>%
  count(clase, sort = TRUE) %>%
  kable(caption = "Cantidad de variables por tipo")
```

Table 1: Cantidad de variables por tipo

clase	n
character	37
integer	27
numeric	15
logical	1

```r
resumen_tipos %>%
  arrange(desc(na_count)) %>%
  slice_head(n = 20) %>%
  kable(caption = "Top 20 variables con mas valores faltantes")
```

Table 2: Top 20 variables con mas valores faltantes

variable	clase	n_unicos	na_count	na_pct
calendar_updated	logical	0	171748	100.00
estimated_revenue_l365d	integer	9746	95502	55.61
price	character	2321	89381	52.04
neighbourhood_group_cleansed	character	17	50683	29.51

variable	clase	n_unicos	na_count	na_pct
review_scores_location	numeric	184	40328	23.48
review_scores_value	numeric	187	40328	23.48
review_scores_checkin	numeric	152	40324	23.48
review_scores_accuracy	numeric	171	40312	23.47
review_scores_communication	numeric	161	40308	23.47
review_scores_cleanliness	numeric	195	40302	23.47
review_scores_rating	numeric	176	40287	23.46
reviews_per_month	numeric	1236	40287	23.46
beds	integer	40	31686	18.45
bathrooms	numeric	38	31396	18.28
license	character	52058	10533	6.13
bedrooms	integer	23	10473	6.10
host_listings_count	integer	394	876	0.51
host_total_listings_count	integer	481	876	0.51
minimum_minimum_nights	integer	185	71	0.04
maximum_minimum_nights	integer	211	71	0.04

```r
if ("price" %in% names(datos) && is.character(datos$price)) {
  datos <- datos %>% mutate(price = readr::parse_number(price))
}

if (!"price" %in% names(datos)) {
  stop("No existe la variable 'price' en el conjunto de datos.")
}
```

En esta base, la variable `price` se encuentra como texto con simbolos monetarios y se convierte a formato numerico para poder construir la variable objetivo y evitar problemas en modelado.

Reutilizacion del particionamiento experimental

```r
detectar_particion_previa <- function(datos_base) {
  resultado <- list(
    encontrada = FALSE,
    fuente = NA_character_,
    train = NULL,
    test = NULL,
    train_index = NULL
  )

  # Caso 1: objetos train/test ya presentes en el ambiente.
  if (exists("train", inherits = FALSE) && exists("test", inherits = FALSE)) {
    tr <- get("train")
    te <- get("test")
    if (is.data.frame(tr) && is.data.frame(te)) {
      resultado$encontrada <- TRUE
      resultado$fuente <- "objetos train/test ya presentes en el ambiente"
      resultado$train <- tr
      resultado$test <- te
      return(resultado)
```

```r
  }
}

# Caso 2: archivos comunes de particion.
candidatos <- list.files(
  path = ".",
  recursive = TRUE,
  full.names = TRUE,
  pattern = "(?i)(train|test|split|particion|partition).*(\\.rds|\\.rda|\\.RData)$"
)

if (length(candidatos) == 0) {
  return(resultado)
}

for (archivo in candidatos) {
  ext <- tolower(tools::file_ext(archivo))

  if (ext == "rds") {
    obj <- tryCatch(readRDS(archivo), error = function(e) NULL)
    if (is.null(obj)) next

    # Formato lista con train/test.
    if (is.list(obj) && all(c("train", "test") %in% names(obj))) {
      if (is.data.frame(obj$train) && is.data.frame(obj$test)) {
        resultado$encontrada <- TRUE
        resultado$fuente <- paste("archivo", archivo, "(lista train/test)")
        resultado$train <- obj$train
        resultado$test <- obj$test
        return(resultado)
      }
    }

    # Formato lista con indice.
    if (is.list(obj) && "train_index" %in% names(obj)) {
      idx <- obj$train_index
      if (is.numeric(idx) && all(idx >= 1) && all(idx <= nrow(datos_base))) {
        resultado$encontrada <- TRUE
        resultado$fuente <- paste("archivo", archivo, "(indice train)")
        resultado$train_index <- idx
        resultado$train <- datos_base[idx, , drop = FALSE]
        resultado$test <- datos_base[-idx, , drop = FALSE]
        return(resultado)
      }
    }
  }

  if (ext %in% c("rda", "rdata")) {
    env_tmp <- new.env(parent = emptyenv())
    cargados <- tryCatch(load(archivo, envir = env_tmp), error = function(e) character(0))
    if (length(cargados) == 0) next

    if (all(c("train", "test") %in% ls(env_tmp))) {
```

```r
        tr <- get("train", envir = env_tmp)
        te <- get("test", envir = env_tmp)
        if (is.data.frame(tr) && is.data.frame(te)) {
          resultado$encontrada <- TRUE
          resultado$fuente <- paste("archivo", archivo, "(objetos train/test)")
          resultado$train <- tr
          resultado$test <- te
          return(resultado)
        }
      }
    }
  }

  resultado
}

if (!"price_category" %in% names(datos)) {
  q1 <- quantile(datos$price, 1 / 3, na.rm = TRUE)
  q2 <- quantile(datos$price, 2 / 3, na.rm = TRUE)

  datos <- datos %>%
    mutate(
      price_category = case_when(
        price <= q1 ~ "barata",
        price <= q2 ~ "media",
        TRUE ~ "cara"
      ),
      price_category = factor(price_category, levels = c("barata", "media", "cara"))
    )
} else {
  datos <- datos %>%
    mutate(price_category = as.factor(price_category))
}

particion_previa <- detectar_particion_previa(datos)

if (isTRUE(particion_previa$encontrada)) {
  train <- particion_previa$train
  test <- particion_previa$test
  fuente_particion <- particion_previa$fuente
} else {
  set.seed(123)
  indice_train <- createDataPartition(datos$price_category, p = 0.8, list = FALSE)

  train <- datos[indice_train, , drop = FALSE]
  test <- datos[-indice_train, , drop = FALSE]

  fuente_particion <- "reconstruida con set.seed(123) y createDataPartition()"

  dir.create("artefactos", showWarnings = FALSE)
  saveRDS(list(train_index = indice_train), file = "artefactos/particion_train_idx.rds")
}
```

```r
cat("Fuente de particionamiento:", fuente_particion, "\n")
```

```
## Fuente de particionamiento: archivo ./artefactos/particion_train_idx.rds (indice train)
```

```r
cat("Dimensiones train:", paste(dim(train), collapse = " x "), "\n")
```

```
## Dimensiones train: 137400 x 81
```

```r
cat("Dimensiones test :", paste(dim(test), collapse = " x "), "\n")
```

```
## Dimensiones test : 34348 x 81
```

```r
prop_train <- prop.table(table(train$price_category))
prop_test <- prop.table(table(test$price_category))

tabla_props <- bind_rows(
  tibble(conjunto = "train", categoria = names(prop_train), proporcion = as.numeric(prop_train)),
  tibble(conjunto = "test", categoria = names(prop_test), proporcion = as.numeric(prop_test))
)

tabla_props %>%
  pivot_wider(names_from = conjunto, values_from = proporcion) %>%
  mutate(diferencia_abs = abs(train - test)) %>%
  kable(digits = 4, caption = "Comparacion de proporciones por categoria de precio")
```

Table 3: Comparacion de proporciones por categoria de precio

categoria	train	test	diferencia_abs
barata	0.1496	0.1496	0
media	0.1465	0.1464	0
cara	0.7040	0.7040	0

Las proporciones por categoria entre entrenamiento y prueba se mantienen comparables, lo cual permite contrastar modelos futuros bajo condiciones experimentales consistentes.

Diagnostico inicial de predictores

```r
glimpse(train)
```

```
## Rows: 137,400
## Columns: 81
## $ id                                          <dbl> 6448, 8502, 22828, 47572,~
## $ listing_url                                 <chr> "https://www.airbnb.com/r~
## $ scrape_id                                   <dbl> 2.025092e+13, 2.025092e+1~
## $ last_scraped                                <chr> "2025-09-17", "2025-09-17~
## $ source                                      <chr> "city scrape", "city scra~
## $ name                                        <chr> "Secluded Studio @ Zilker~
```

```
## $ description                                  <chr> "Clean, private space wit~
## $ neighborhood_overview                        <chr> "The neighborhood is fun ~
## $ picture_url                                  <chr> "https://a0.muscache.com/~
## $ host_id                                      <int> 14156, 25298, 56488, 2159~
## $ host_url                                     <chr> "https://www.airbnb.com/u~
## $ host_name                                    <chr> "Amy", "Karen", "David", ~
## $ host_since                                   <chr> "2009-04-20", "2009-07-11~
## $ host_location                                <chr> "Austin, TX", "Austin, TX~
## $ host_about                                   <chr> "We are a family of four ~
## $ host_response_time                           <chr> "within an hour", "within~
## $ host_response_rate                           <chr> "100%", "100%", "100%", "~
## $ host_acceptance_rate                         <chr> "96%", "60%", "71%", "N/A~
## $ host_is_superhost                            <chr> "t", "f", "t", "f", "f", ~
## $ host_thumbnail_url                           <chr> "https://a0.muscache.com/~
## $ host_picture_url                             <chr> "https://a0.muscache.com/~
## $ host_neighbourhood                           <chr> "Zilker", "East Riverside~
## $ host_listings_count                          <int> 1, 1, 1, 1, 1, 1, 1, 1, 2~
## $ host_total_listings_count                    <int> 2, 1, 1, 1, 2, 2, 2, 1, 5~
## $ host_verifications                           <chr> "['email', 'phone']", "['~
## $ host_has_profile_pic                         <chr> "t", "t", "t", "t", "t", ~
## $ host_identity_verified                       <chr> "t", "f", "t", "t", "f", ~
## $ neighbourhood                                <chr> "Neighborhood highlights"~
## $ neighbourhood_cleansed                       <chr> "78704", "78741", "78741"~
## $ neighbourhood_group_cleansed                 <chr> NA, NA, NA, NA, NA, NA, N~
## $ latitude                                     <dbl> 30.26034, 30.23466, 30.23~
## $ longitude                                    <dbl> -97.76487, -97.73682, -97~
## $ property_type                                <chr> "Entire guesthouse", "Ent~
## $ room_type                                    <chr> "Entire home/apt", "Entir~
## $ accommodates                                 <int> 2, 2, 2, 1, 2, 6, 3, 2, 2~
## $ bathrooms                                    <dbl> 1.0, 1.0, 1.0, 1.0, 1.0, ~
## $ bathrooms_text                               <chr> "1 bath", "1 bath", "1 ba~
## $ bedrooms                                     <int> 1, 1, 1, 1, 2, 1, 1, 1, 1~
## $ beds                                         <int> 2, 1, 1, 1, 0, 3, 1, 1, 1~
## $ amenities                                    <chr> "[\"Private patio or balc~
## $ price                                        <dbl> 160, 38, 58, 49, 300, 150~
## $ minimum_nights                               <int> 3, 4, 30, 30, 7, 1, 2, 3,~
## $ maximum_nights                               <int> 365, 90, 365, 120, 365, 3~
## $ minimum_minimum_nights                       <int> 3, 3, 30, 30, 7, 1, 2, 3,~
## $ maximum_minimum_nights                       <int> 3, 14, 30, 30, 7, 5, 2, 3~
## $ minimum_maximum_nights                       <int> 1125, 90, 365, 120, 365, ~
## $ maximum_maximum_nights                       <int> 1125, 90, 365, 120, 365, ~
## $ minimum_nights_avg_ntm                       <dbl> 3.0, 6.0, 30.0, 30.0, 7.0~
## $ maximum_nights_avg_ntm                       <dbl> 1125.0, 90.0, 365.0, 120.~
## $ calendar_updated                             <lgl> NA, NA, NA, NA, NA, NA, N~
## $ has_availability                             <chr> "t", "t", "t", "t", "t", ~
## $ availability_30                              <int> 12, 29, 27, 0, 30, 8, 18,~
## $ availability_60                              <int> 30, 59, 40, 0, 60, 38, 46~
## $ availability_90                              <int> 50, 88, 40, 0, 90, 68, 75~
## $ availability_365                             <int> 316, 88, 211, 258, 365, 3~
## $ calendar_last_scraped                        <chr> "2025-09-17", "2025-09-17~
## $ number_of_reviews                            <int> 339, 54, 56, 9, 2, 1090, ~
## $ number_of_reviews_ltm                        <int> 14, 1, 3, 0, 0, 26, 2, 0,~
## $ number_of_reviews_l30d                       <int> 1, 0, 0, 0, 0, 0, 0, 0, 0~
## $ availability_eoy                             <int> 61, 88, 40, 0, 106, 84, 7~
```

```
## $ number_of_reviews_ly                        <int> 20, 2, 3, 0, 0, 56, 7, 0,~
## $ estimated_occupancy_l365d                   <int> 84, 8, 180, 0, 0, 156, 12~
## $ estimated_revenue_l365d                     <int> 13440, 304, 10440, 0, 0, ~
## $ first_review                                <chr> "2011-09-06", "2010-02-19~
## $ last_review                                 <chr> "2025-08-20", "2025-05-05~
## $ review_scores_rating                        <dbl> 4.97, 4.57, 4.95, 5.00, 5~
## $ review_scores_accuracy                      <dbl> 4.97, 4.55, 5.00, 5.00, 5~
## $ review_scores_cleanliness                   <dbl> 4.96, 4.67, 4.98, 5.00, 5~
## $ review_scores_checkin                       <dbl> 4.99, 4.85, 5.00, 5.00, 5~
## $ review_scores_communication                 <dbl> 4.98, 4.88, 5.00, 5.00, 5~
## $ review_scores_location                      <dbl> 4.97, 4.69, 4.72, 5.00, 5~
## $ review_scores_value                         <dbl> 4.88, 4.63, 4.84, 5.00, 5~
## $ license                                     <chr> NA, NA, NA, NA, NA, NA, N~
## $ instant_bookable                            <chr> "t", "f", "f", "f", "f", ~
## $ calculated_host_listings_count              <int> 1, 1, 1, 1, 1, 1, 1, 1, 2~
## $ calculated_host_listings_count_entire_homes  <int> 1, 1, 1, 0, 1, 1, 1, 1, 2~
## $ calculated_host_listings_count_private_rooms <int> 0, 0, 0, 1, 0, 0, 0, 0, 0~
## $ calculated_host_listings_count_shared_rooms  <int> 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ reviews_per_month                           <dbl> 1.98, 0.28, 0.30, 0.11, 0~
## $ city                                        <chr> "Austin, Texas", "Austin,~
## $ price_category                              <fct> media, barata, barata, ba~
```

```r
resumen_numericas <- train %>%
  summarise(across(where(is.numeric), list(
    media = ~ mean(.x, na.rm = TRUE),
    sd = ~ sd(.x, na.rm = TRUE),
    min = ~ min(.x, na.rm = TRUE),
    max = ~ max(.x, na.rm = TRUE)
  )))

resumen_na <- train %>%
  summarise(across(everything(), ~ sum(is.na(.x)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "na_count") %>%
  mutate(na_pct = 100 * na_count / nrow(train)) %>%
  arrange(desc(na_count))

perfil_categoricas <- train %>%
  select(where(~ is.character(.x) || is.factor(.x))) %>%
  summarise(across(everything(), ~ n_distinct(.x, na.rm = TRUE))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "n_categorias") %>%
  arrange(desc(n_categorias))

resumen_na %>%
  slice_head(n = 20) %>%
  kable(caption = "Variables con mayor cantidad de NA en entrenamiento")
```

Table 4: Variables con mayor cantidad de NA en entrenamiento

variable	na_count	na_pct
calendar_updated	137400	100.0000000
price	76466	55.6521106
estimated_revenue_l365d	76466	55.6521106

variable	na_count	na_pct
neighbourhood_group_cleansed	40600	29.5487627
review_scores_value	32269	23.4854440
review_scores_location	32268	23.4847162
review_scores_checkin	32265	23.4825328
review_scores_accuracy	32255	23.4752547
review_scores_communication	32251	23.4723435
review_scores_cleanliness	32247	23.4694323
review_scores_rating	32237	23.4621543
reviews_per_month	32237	23.4621543
beds	25356	18.4541485
bathrooms	25120	18.2823872
license	8455	6.1535662
bedrooms	8342	6.0713246
host_listings_count	701	0.5101892
host_total_listings_count	701	0.5101892
minimum_minimum_nights	54	0.0393013
maximum_minimum_nights	54	0.0393013

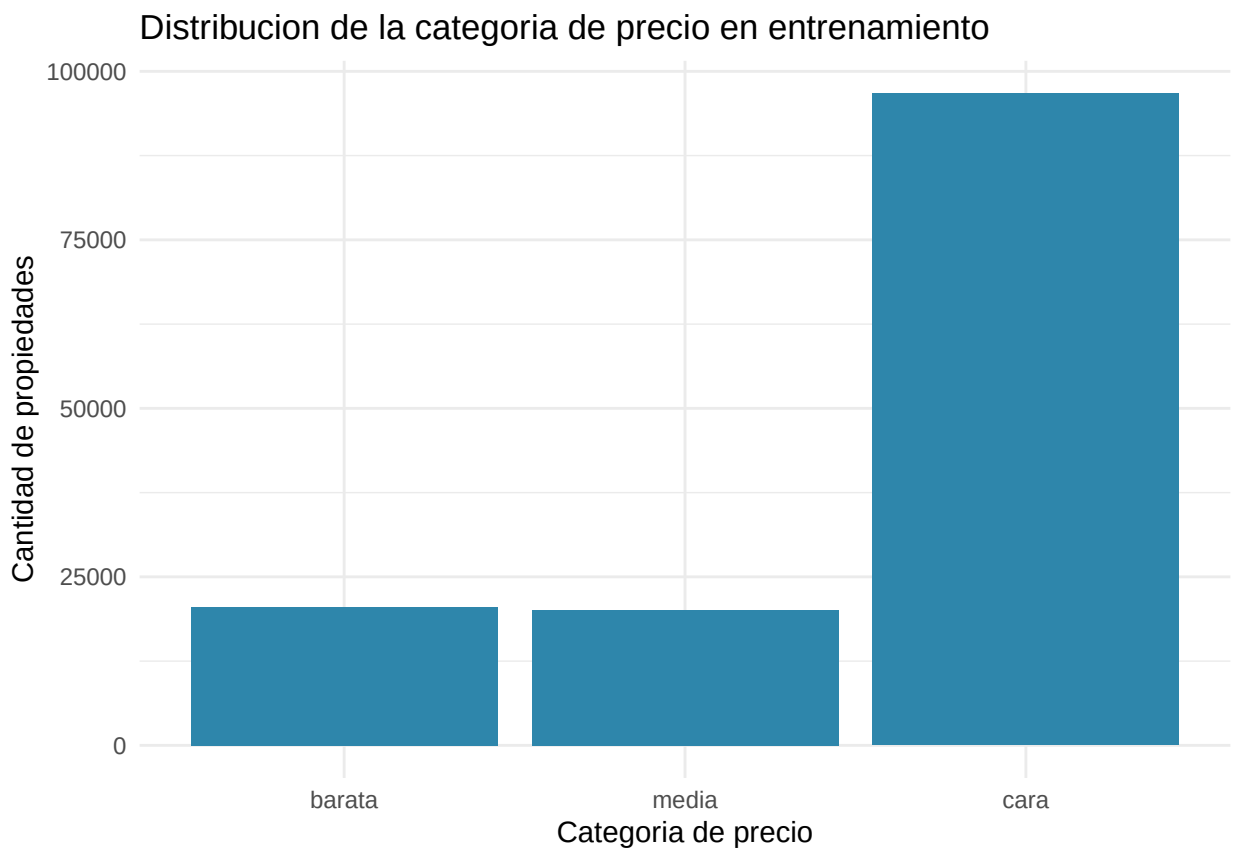
```r
perfil_categoricas %>%
  slice_head(n = 20) %>%
  kable(caption = "Variables categoricas con mayor cardinalidad")
```

Table 5: Variables categoricas con mayor cardinalidad

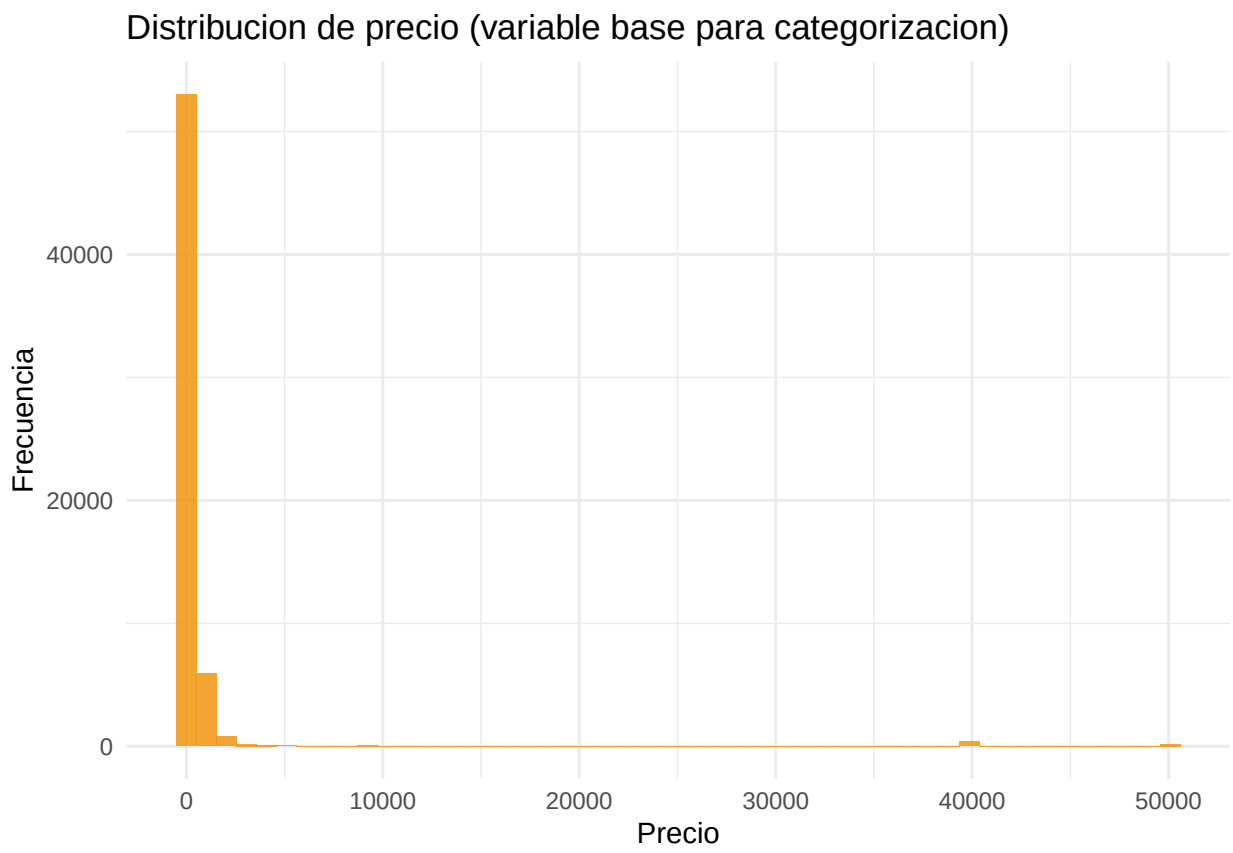
variable	n_categorias
listing_url	137400
picture_url	134531
name	131353
amenities	121138
description	115584
host_url	66581
host_picture_url	64441
host_thumbnail_url	64415
neighborhood_overview	50122
license	42592
host_about	35387
host_name	19852
host_since	5929
first_review	5171
last_review	3795
host_neighbourhood	2677
host_location	2574
neighbourhood_cleansed	853
property_type	110
host_acceptance_rate	103

```r
ggplot(train, aes(x = price_category)) +
  geom_bar(fill = "#2E86AB") +
  labs(
```

```
    title = "Distribucion de la categoria de precio en entrenamiento",
    x = "Categoria de precio",
    y = "Cantidad de propiedades"
  ) +
  theme_minimal()
```




```
ggplot(train, aes(x = price)) +
  geom_histogram(bins = 50, fill = "#F18F01", alpha = 0.8) +
  labs(
    title = "Distribucion de precio (variable base para categorizacion)",
    x = "Precio",
    y = "Frecuencia"
  ) +
  theme_minimal()
```



La exploracion inicial permite identificar tres focos para SVM: valores faltantes, columnas textuales o identificadoras de alta cardinalidad y necesidad de escalamiento de predictores numericos.

Transformaciones para modelado SVM

```
# Limpieza minima de la variable objetivo.
train <- train %>% mutate(price_category = factor(price_category, levels = c("barata", "media", "cara"))
test <- test %>% mutate(price_category = factor(price_category, levels = levels(train$price_category)))

# Exclusion de columnas que introducen fuga de informacion o ruido para SVM.
patrones_id <- "(^id$|_id$|url|name$|description|about|picture|thumbnail|host_url|listing_url|scrape_id
cols_patron <- names(train)[str_detect(names(train), regex(patrones_id, ignore_case = TRUE))]

cols_all_na <- names(train)[sapply(train, function(x) all(is.na(x)))]

# Variables categoricas de cardinalidad extrema.
cat_cols <- names(train)[sapply(train, function(x) is.character(x) || is.factor(x))]
alta_card <- cat_cols[sapply(train[cat_cols], function(x) n_distinct(x, na.rm = TRUE) > 100)]

columnas_excluir <- unique(c("price", cols_patron, cols_all_na, alta_card))
columnas_excluir <- setdiff(columnas_excluir, "price_category")

train_model <- train %>% select(-any_of(columnas_excluir))
test_model <- test %>% select(-any_of(columnas_excluir))
```

```r
# Separacion de X e y.
y_train <- train_model$price_category
y_test <- test_model$price_category

x_train_base <- train_model %>% select(-price_category)
x_test_base <- test_model %>% select(-price_category)

# Conversion de categoricas a dummies.
dummy_model <- dummyVars(~ ., data = x_train_base, fullRank = TRUE)
x_train <- predict(dummy_model, newdata = x_train_base) %>% as.data.frame()
x_test <- predict(dummy_model, newdata = x_test_base) %>% as.data.frame()

# Alineacion estricta de columnas entre train y test.
cols_faltantes_test <- setdiff(names(x_train), names(x_test))
if (length(cols_faltantes_test) > 0) {
  for (col in cols_faltantes_test) x_test[[col]] <- 0
}

cols_extra_test <- setdiff(names(x_test), names(x_train))
if (length(cols_extra_test) > 0) {
  x_test <- x_test %>% select(-any_of(cols_extra_test))
}

x_test <- x_test[, names(x_train), drop = FALSE]

# Imputacion y escalamiento para SVM.
preprocess_model <- preProcess(x_train, method = c("medianImpute", "center", "scale"))
x_train_scaled <- predict(preprocess_model, x_train)
x_test_scaled <- predict(preprocess_model, x_test)

# Reconstruccion final para entrenamiento y evaluacion.
train_svm <- x_train_scaled %>% mutate(price_category = y_train)
test_svm <- x_test_scaled %>% mutate(price_category = y_test)

train_svm$price_category <- factor(train_svm$price_category, levels = c("barata", "media", "cara"))
test_svm$price_category <- factor(test_svm$price_category, levels = levels(train_svm$price_category))
```

```r
cat("Dimensiones train_svm:", paste(dim(train_svm), collapse = " x "), "\n")
```

```
## Dimensiones train_svm: 137400 x 247
```

```r
cat("Dimensiones test_svm :", paste(dim(test_svm), collapse = " x "), "\n")
```

```
## Dimensiones test_svm : 34348 x 247
```

```r
table(train_svm$price_category)
```

```
##
## barata  media   cara
##  20552  20123  96725
```

```r
table(test_svm$price_category)
```

```
##
## barata  media   cara
##   5137   5030  24181
```

```r
cat("¿Hay NA en train_svm?:", anyNA(train_svm), "\n")
```

```
## ¿Hay NA en train_svm?: FALSE
```

```r
cat("¿Hay NA en test_svm?:", anyNA(test_svm), "\n")
```

```
## ¿Hay NA en test_svm?: FALSE
```

```r
dir.create("artefactos", showWarnings = FALSE)

saveRDS(train_svm, "artefactos/train_svm.rds")
saveRDS(test_svm, "artefactos/test_svm.rds")
saveRDS(dummy_model, "artefactos/dummy_model_svm.rds")
saveRDS(preprocess_model, "artefactos/preprocess_model_svm.rds")
```

Cierre del avance

Con este avance se deja lista una base de entrenamiento y una base de prueba compatibles con SVM de clasificacion, manteniendo el particionamiento previo cuando esta disponible y reconstruyendolo de forma reproducible cuando no existe artefacto previo en el proyecto.

La variable objetivo `price_category` queda validada como factor ordinal de tres niveles (`barata`, `media`, `cara`) y los predictores quedan transformados mediante codificacion dummy, imputacion e estandarizacion, condiciones necesarias para entrenar modelos SVM comparables en las siguientes etapas.

Diseno experimental de los modelos SVM

```r
train_svm <- readRDS("artefactos/train_svm.rds")
test_svm  <- readRDS("artefactos/test_svm.rds")

str(train_svm[, c(ncol(train_svm) - 2, ncol(train_svm) - 1, ncol(train_svm))])
```

```
## 'data.frame':    137400 obs. of  3 variables:
##  $ citySan Francisco, California: num  -0.214 -0.214 -0.214 -0.214 -0.214 ...
##  $ cityWashington, D.C.         : num  -0.196 -0.196 -0.196 -0.196 -0.196 ...
##  $ price_category               : Factor w/ 3 levels "barata","media",..: 2 1 1 1 3 2 2 1 1 1 ...
```

```r
str(test_svm[, c(ncol(test_svm) - 2, ncol(test_svm) - 1, ncol(test_svm))])
```

```
## 'data.frame':    34348 obs. of  3 variables:
##  $ citySan Francisco, California: num  -0.214 -0.214 -0.214 -0.214 -0.214 ...
##  $ cityWashington, D.C.         : num  -0.196 -0.196 -0.196 -0.196 -0.196 ...
##  $ price_category               : Factor w/ 3 levels "barata","media",..: 1 2 2 1 1 1 2 3 3 3 ...
```

```r
cat("DistribuciOn en entrenamiento:\n")
```

```
## DistribuciOn en entrenamiento:
```

```r
print(table(train_svm$price_category))
```

```
##
## barata  media   cara
##  20552  20123  96725
```

```r
cat("DistribuciOn en prueba:\n")
```

```
## DistribuciOn en prueba:
```

```r
print(table(test_svm$price_category))
```

```
##
## barata  media   cara
##   5137   5030  24181
```

```r
cat("NA en train_svm:", sum(is.na(train_svm)), "\n")
```

```
## NA en train_svm: 0
```

```r
cat("NA en test_svm: ", sum(is.na(test_svm)),  "\n")
```

```
## NA en test_svm:  0
```

```r
train_svm$price_category <- as.factor(train_svm$price_category)
test_svm$price_category  <- as.factor(test_svm$price_category)
```

```r
x_train <- train_svm %>% select(-price_category)
y_train <- train_svm$price_category

x_test <- test_svm %>% select(-price_category)
y_test <- test_svm$price_category

dif_1 <- setdiff(names(x_train), names(x_test))
dif_2 <- setdiff(names(x_test),  names(x_train))

if (length(dif_1) > 0 || length(dif_2) > 0) {
  warning("Las columnas de train y test no coinciden. Revisar preprocesamiento.")
} else {
  cat("Las columnas de entrenamiento y prueba son identicas.\n")
}
```

```
## Las columnas de entrenamiento y prueba son identicas.
```

Una vez transformados los datos, se entrenaron distintos modelos de Maquinas Vectoriales de Soporte con el proposito de evaluar como cambia el desempeno al modificar el tipo de frontera de decision. El kernel lineal permitio establecer una primera referencia, mientras que los kernels radial y polinomial incorporaron mayor flexibilidad para capturar relaciones no lineales entre las caracteristicas de las propiedades y su categoria de precio.

Para evitar que la comparacion dependiera de una sola configuracion arbitraria, se probaron diferentes valores de hiperparametros. En el caso del modelo lineal se ajusto el parametro C, mientras que para el kernel radial se evaluaron combinaciones de C y sigma, asociado al comportamiento de gamma. Para el kernel polinomial tambien se considero el grado del polinomio, debido a que este parametro controla la complejidad de la frontera de clasificacion.

```r
set.seed(123)
dir.create("artefactos", showWarnings = FALSE, recursive = TRUE)

# Cambia a FALSE si quieres volver al ajuste completo.
modo_rapido_knit <- TRUE
folds_svm <- if (isTRUE(modo_rapido_knit)) 3 else 5
guardar_predicciones_cv <- if (isTRUE(modo_rapido_knit)) "none" else "final"

control_svm <- trainControl(
  method          = "cv",
  number          = folds_svm,
  classProbs      = FALSE,
  savePredictions = guardar_predicciones_cv
)
```

Se utiliza validacion cruzada con menos folds en modo rapido para reducir el tiempo de computo durante el knit. Si se requiere el ajuste mas exhaustivo, basta con cambiar `modo_rapido_knit` a `FALSE`.

Evaluacion de kernels para clasificacion de precios

Kernel lineal

```r
grid_lineal <- if (isTRUE(modo_rapido_knit)) {
  expand.grid(C = 1)
} else {
  expand.grid(C = c(0.1, 1, 10))
}

control_svm_lineal <- if (isTRUE(modo_rapido_knit)) {
  trainControl(
    method = "none",
    classProbs = FALSE,
    savePredictions = "none"
  )
} else {
  control_svm
}

ruta_modelo_lineal <- if (isTRUE(modo_rapido_knit)) {
  "artefactos/modelo_svm_lineal_rapido.rds"
```

```r
} else {
  "artefactos/modelo_svm_lineal.rds"
}

if (file.exists(ruta_modelo_lineal)) {
  modelo_svm_lineal <- readRDS(ruta_modelo_lineal)
  tiempo_lineal <- c(elapsed = NA_real_)
} else {
  x_train_lineal <- x_train
  y_train_lineal <- y_train

  if (isTRUE(modo_rapido_knit)) {
    n_max_lineal <- min(3000, nrow(x_train))
    idx_lineal <- sample(seq_len(nrow(x_train)), size = n_max_lineal)
    x_train_lineal <- x_train[idx_lineal, , drop = FALSE]
    y_train_lineal <- y_train[idx_lineal]
  }

  set.seed(123)
  tiempo_lineal <- system.time({
    modelo_svm_lineal <- train(
      x         = x_train_lineal,
      y         = y_train_lineal,
      method    = "svmLinear",
      trControl = control_svm_lineal,
      tuneGrid  = grid_lineal,
      metric    = "Accuracy"
    )
  })
  saveRDS(modelo_svm_lineal, ruta_modelo_lineal)
}

modelo_svm_lineal
```

```
## Support Vector Machines with Linear Kernel
##
## 3000 samples
##  246 predictor
##    3 classes: 'barata', 'media', 'cara'
##
## No pre-processing
## Resampling: None
```

```r
if (!isTRUE(modo_rapido_knit)) {
  plot(modelo_svm_lineal, main = "Accuracy vs. C - kernel lineal")
}
```

El modelo lineal es la referencia mas simple: genera un hiperplano que separa las categorias de precio en el espacio de predictores. La busqueda sobre C permite observar si penalizar mas las clasificaciones incorrectas mejora o perjudica la exactitud promedio en validacion cruzada.

Kernel radial

```r
grid_radial <- if (isTRUE(modo_rapido_knit)) {
  expand.grid(C = 1, sigma = 0.05)
} else {
  expand.grid(
    C     = c(0.1, 1, 10),
    sigma = c(0.01, 0.05, 0.1)
  )
}

control_svm_radial <- if (isTRUE(modo_rapido_knit)) {
  trainControl(
    method = "none",
    classProbs = FALSE,
    savePredictions = "none"
  )
} else {
  control_svm
}

ruta_modelo_radial <- if (isTRUE(modo_rapido_knit)) {
  "artefactos/modelo_svm_radial_rapido.rds"
} else {
  "artefactos/modelo_svm_radial.rds"
}

if (file.exists(ruta_modelo_radial)) {
  modelo_svm_radial <- readRDS(ruta_modelo_radial)
  tiempo_radial <- c(elapsed = NA_real_)
} else {
  x_train_radial <- x_train
  y_train_radial <- y_train

  if (isTRUE(modo_rapido_knit)) {
    n_max_radial <- min(3000, nrow(x_train))
    idx_radial <- sample(seq_len(nrow(x_train)), size = n_max_radial)
    x_train_radial <- x_train[idx_radial, , drop = FALSE]
    y_train_radial <- y_train[idx_radial]
  }

  set.seed(123)
  tiempo_radial <- system.time({
    modelo_svm_radial <- train(
      x         = x_train_radial,
      y         = y_train_radial,
      method    = "svmRadial",
      trControl = control_svm_radial,
      tuneGrid  = grid_radial,
      metric    = "Accuracy"
    )
  })
  saveRDS(modelo_svm_radial, ruta_modelo_radial)
```

```r
}

modelo_svm_radial
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 3000 samples
##  246 predictor
##    3 classes: 'barata', 'media', 'cara'
##
## No pre-processing
## Resampling: None
```

```r
if (!isTRUE(modo_rapido_knit)) {
  plot(modelo_svm_radial, main = "Accuracy vs. C y sigma - kernel radial")
}
```

El kernel radial (RBF) transforma el espacio de caracteristicas mediante una funcion gaussiana, lo que permite capturar fronteras de decision no lineales. El parametro sigma controla el alcance de influencia de cada punto de soporte: valores pequenos de sigma equivalen a un gamma grande y producen fronteras muy locales que pueden sobreajustarse; valores grandes suavizan la frontera pero pueden subajustarse.

Kernel polinomial

```r
grid_polinomial <- if (isTRUE(modo_rapido_knit)) {
  expand.grid(degree = 2, scale = 0.05, C = 1)
} else {
  expand.grid(
    degree = c(2, 3),
    scale  = c(0.01, 0.05, 0.1),
    C      = c(0.1, 1, 10)
  )
}

control_svm_polinomial <- if (isTRUE(modo_rapido_knit)) {
  trainControl(
    method = "none",
    classProbs = FALSE,
    savePredictions = "none"
  )
} else {
  control_svm
}

ruta_modelo_polinomial <- if (isTRUE(modo_rapido_knit)) {
  "artefactos/modelo_svm_polinomial_rapido.rds"
} else {
  "artefactos/modelo_svm_polinomial.rds"
}

if (file.exists(ruta_modelo_polinomial)) {
```

```r
  modelo_svm_polinomial <- readRDS(ruta_modelo_polinomial)
  tiempo_polinomial <- c(elapsed = NA_real_)
} else {
  x_train_polinomial <- x_train
  y_train_polinomial <- y_train

  if (isTRUE(modo_rapido_knit)) {
    n_max_polinomial <- min(3000, nrow(x_train))
    idx_polinomial <- sample(seq_len(nrow(x_train)), size = n_max_polinomial)
    x_train_polinomial <- x_train[idx_polinomial, , drop = FALSE]
    y_train_polinomial <- y_train[idx_polinomial]
  }

  set.seed(123)
  tiempo_polinomial <- system.time({
    modelo_svm_polinomial <- train(
      x         = x_train_polinomial,
      y         = y_train_polinomial,
      method    = "svmPoly",
      trControl = control_svm_polinomial,
      tuneGrid  = grid_polinomial,
      metric    = "Accuracy"
    )
  })
  saveRDS(modelo_svm_polinomial, ruta_modelo_polinomial)
}

modelo_svm_polinomial
```

```
## Support Vector Machines with Polynomial Kernel
##
## 3000 samples
##  246 predictor
##    3 classes: 'barata', 'media', 'cara'
##
## No pre-processing
## Resampling: None
```

```r
if (!isTRUE(modo_rapido_knit)) {
  plot(modelo_svm_polinomial, main = "Accuracy vs. hiperparametros - kernel polinomial")
}
```

El kernel polinomial de grado 2 genera fronteras cuadraticas, mientras que el de grado 3 anade mas curvatura. Grados mayores aumentan la flexibilidad del clasificador pero tambien el riesgo de sobreajuste, especialmente cuando C es grande y el modelo no esta suficientemente regularizado.

Prediccion de categorias de precio

```r
pred_lineal     <- predict(modelo_svm_lineal,     newdata = x_test)
pred_radial     <- predict(modelo_svm_radial,     newdata = x_test)
```

```r
pred_polinomial <- predict(modelo_svm_polinomial, newdata = x_test)

cat("DistribuciOn real en prueba:\n")
```

```
## DistribuciOn real en prueba:
```

```r
print(table(y_test))
```

```
## y_test
## barata  media   cara
##   5137   5030  24181
```

```r
cat("\nPredicciones - kernel lineal:\n")
```

```
##
## Predicciones - kernel lineal:
```

```r
print(table(pred_lineal))
```

```
## pred_lineal
## barata  media   cara
##   5259   5271  23818
```

```r
cat("\nPredicciones - kernel radial:\n")
```

```
##
## Predicciones - kernel radial:
```

```r
print(table(pred_radial))
```

```
## pred_radial
## barata  media   cara
##   2851   2972  28525
```

```r
cat("\nPredicciones - kernel polinomial:\n")
```

```
##
## Predicciones - kernel polinomial:
```

```r
print(table(pred_polinomial))
```

```
## pred_polinomial
## barata  media   cara
##   5431   5238  23679
```

Primer analisis comparativo de desempeno

```r
tabla_tiempos <- tibble(
  modelo           = c("SVM lineal", "SVM radial", "SVM polinomial"),
  tiempo_segundos  = c(
    tiempo_lineal["elapsed"],
    tiempo_radial["elapsed"],
    tiempo_polinomial["elapsed"]
  ),
  mejor_accuracy_cv = c(
    max(modelo_svm_lineal$results$Accuracy),
    max(modelo_svm_radial$results$Accuracy),
    max(modelo_svm_polinomial$results$Accuracy)
  )
)

kable(tabla_tiempos, digits = 4,
      caption = "Comparacion preliminar de modelos SVM: tiempo de entrenamiento y exactitud en validaci
```

Table 6: Comparacion preliminar de modelos SVM: tiempo de entrenamiento y exactitud en validacion cruzada

modelo	tiempo_segundos	mejor_accuracy_cv
SVM lineal	NA	-Inf
SVM radial	NA	-Inf
SVM polinomial	NA	-Inf

```r
acc_lineal     <- mean(pred_lineal     == y_test)
acc_radial     <- mean(pred_radial     == y_test)
acc_polinomial <- mean(pred_polinomial == y_test)

tabla_acc_prueba <- tibble(
  modelo     = c("SVM lineal", "SVM radial", "SVM polinomial"),
  accuracy   = c(acc_lineal, acc_radial, acc_polinomial)
)

kable(tabla_acc_prueba, digits = 4,
      caption = "Exactitud de cada modelo sobre el conjunto de prueba")
```

Table 7: Exactitud de cada modelo sobre el conjunto de prueba

modelo	accuracy
SVM lineal	0.8771
SVM radial	0.7833
SVM polinomial	0.8327

```r
# Los modelos ya se guardan dentro de cada chunk de entrenamiento si no existian.
# Este bloque solo confirma que los artefactos estan presentes.
cat("modelo_svm_lineal.rds:    ", file.exists("artefactos/modelo_svm_lineal.rds"), "\n")
```

```
## modelo_svm_lineal.rds:     FALSE
```

```r
cat("modelo_svm_radial.rds:    ", file.exists("artefactos/modelo_svm_radial.rds"), "\n")
```

```
## modelo_svm_radial.rds:     FALSE
```

```r
cat("modelo_svm_polinomial.rds:", file.exists("artefactos/modelo_svm_polinomial.rds"), "\n")
```

```
## modelo_svm_polinomial.rds: FALSE
```

Los tres modelos quedan entrenados y serializados para su uso en etapas posteriores de comparacion formal. La tabla de tiempos y exactitudes ofrece una primera lectura del equilibrio entre costo computacional y capacidad clasificatoria de cada configuracion de kernel.

Evaluacion de la clasificacion en el conjunto de prueba

Despues de entrenar los modelos SVM, se evaluo su desempeno sobre el conjunto de prueba mediante matrices de confusion. Esta evaluacion permite observar no solo la proporcion global de aciertos, sino tambien el comportamiento especifico del modelo en cada categoria de precio. Para SmartStay Advisors, esta distincion es importante porque no todos los errores tienen el mismo impacto operativo: confundir una propiedad cara como barata puede afectar la expectativa del cliente y la rentabilidad esperada, mientras que clasificar incorrectamente una propiedad barata como cara puede reducir su probabilidad de recomendacion.

```r
matriz_lineal     <- confusionMatrix(pred_lineal,     y_test)
matriz_radial     <- confusionMatrix(pred_radial,     y_test)
matriz_polinomial <- confusionMatrix(pred_polinomial, y_test)

matriz_lineal
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media  cara
##     barata   4063   896   300
##     media     919  3268  1084
##     cara      155   866 22797
##
## Overall Statistics
##
##                Accuracy : 0.8771
##                  95% CI : (0.8736, 0.8806)
##     No Information Rate : 0.704
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7366
##
##  Mcnemar's Test P-Value : 2.777e-15
##
## Statistics by Class:
##
##                      Class: barata Class: media Class: cara
## Sensitivity                 0.7909      0.64970      0.9428
```

```
## Specificity                 0.9591      0.93168      0.8996
## Pos Pred Value              0.7726      0.62000      0.9571
## Neg Pred Value              0.9631      0.93940      0.8686
## Prevalence                  0.1496      0.14644      0.7040
## Detection Rate              0.1183      0.09514      0.6637
## Detection Prevalence        0.1531      0.15346      0.6934
## Balanced Accuracy           0.8750      0.79069      0.9212
```

```
matriz_radial
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media  cara
##     barata   1953   727   171
##     media     653  1631   688
##     cara     2531  2672 23322
##
## Overall Statistics
##
##                Accuracy : 0.7833
##                  95% CI : (0.7789, 0.7877)
##     No Information Rate : 0.704
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4448
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: media Class: cara
## Sensitivity               0.38018      0.32425      0.9645
## Specificity               0.96926      0.95426      0.4882
## Pos Pred Value            0.68502      0.54879      0.8176
## Neg Pred Value            0.89891      0.89167      0.8525
## Prevalence                0.14956      0.14644      0.7040
## Detection Rate            0.05686      0.04748      0.6790
## Detection Prevalence      0.08300      0.08653      0.8305
## Balanced Accuracy         0.67472      0.63926      0.7264
```

```
matriz_polinomial
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata media  cara
##     barata   3610  1305   516
##     media    1239  2663  1336
##     cara      288  1062 22329
##
## Overall Statistics
##
```

```
##                Accuracy : 0.8327
##                  95% CI : (0.8287, 0.8366)
##     No Information Rate : 0.704
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.6431
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: media Class: cara
## Sensitivity                 0.7027      0.52942      0.9234
## Specificity                 0.9377      0.91217      0.8672
## Pos Pred Value              0.6647      0.50840      0.9430
## Neg Pred Value              0.9472      0.91869      0.8264
## Prevalence                  0.1496      0.14644      0.7040
## Detection Rate              0.1051      0.07753      0.6501
## Detection Prevalence        0.1581      0.15250      0.6894
## Balanced Accuracy           0.8202      0.72080      0.8953
```

```r
accuracy_lineal     <- matriz_lineal$overall["Accuracy"]
accuracy_radial     <- matriz_radial$overall["Accuracy"]
accuracy_polinomial <- matriz_polinomial$overall["Accuracy"]

kappa_lineal     <- matriz_lineal$overall["Kappa"]
kappa_radial     <- matriz_radial$overall["Kappa"]
kappa_polinomial <- matriz_polinomial$overall["Kappa"]
```

Analisis de errores por categoria de precio

Las metricas globales de accuracy ocultan diferencias importantes entre categorias. Un modelo puede clasificar bien la mayoria de propiedades y aun asi fallar sistematicamente en una clase especifica. Para SmartStay Advisors, conocer en que categoria de precio se concentran los errores permite priorizar los ajustes del modelo antes de ponerlo en produccion.

```r
metricas_lineal     <- as.data.frame(matriz_lineal$byClass)
metricas_radial     <- as.data.frame(matriz_radial$byClass)
metricas_polinomial <- as.data.frame(matriz_polinomial$byClass)

cat("Metricas por clase - kernel lineal:\n")
```

```
## Metricas por clase - kernel lineal:
```

```r
kable(metricas_lineal[, c("Sensitivity", "Specificity", "Pos Pred Value", "Balanced Accuracy")],
      digits = 4, caption = "Desempeno por categoria de precio - SVM lineal")
```

Table 8: Desempeno por categoria de precio — SVM lineal

	Sensitivity	Specificity	Pos Pred Value	Balanced Accuracy
Class: barata	0.7909	0.9591	0.7726	0.8750
Class: media	0.6497	0.9317	0.6200	0.7907
Class: cara	0.9428	0.8996	0.9571	0.9212

```r
cat("Metricas por clase - kernel radial:\n")
```

```
## Metricas por clase - kernel radial:
```

```r
kable(metricas_radial[, c("Sensitivity", "Specificity", "Pos Pred Value", "Balanced Accuracy")],
      digits = 4, caption = "Desempeno por categoria de precio - SVM radial")
```

Table 9: Desempeno por categoria de precio — SVM radial

	Sensitivity	Specificity	Pos Pred Value	Balanced Accuracy
Class: barata	0.3802	0.9693	0.6850	0.6747
Class: media	0.3243	0.9543	0.5488	0.6393
Class: cara	0.9645	0.4882	0.8176	0.7264

```r
cat("Metricas por clase - kernel polinomial:\n")
```

```
## Metricas por clase - kernel polinomial:
```

```r
kable(metricas_polinomial[, c("Sensitivity", "Specificity", "Pos Pred Value", "Balanced Accuracy")],
      digits = 4, caption = "Desempeno por categoria de precio - SVM polinomial")
```

Table 10: Desempeno por categoria de precio — SVM polinomial

	Sensitivity	Specificity	Pos Pred Value	Balanced Accuracy
Class: barata	0.7027	0.9377	0.6647	0.8202
Class: media	0.5294	0.9122	0.5084	0.7208
Class: cara	0.9234	0.8672	0.9430	0.8953

```r
tabla_metricas_svm <- tibble(
  modelo   = c("SVM lineal", "SVM radial", "SVM polinomial"),
  accuracy = c(accuracy_lineal, accuracy_radial, accuracy_polinomial),
  kappa    = c(kappa_lineal,    kappa_radial,    kappa_polinomial)
)

tabla_resumen_svm <- tabla_metricas_svm %>%
  left_join(tabla_tiempos, by = "modelo")

kable(tabla_resumen_svm, digits = 4,
      caption = "Resumen comparativo de modelos SVM: accuracy, kappa y tiempo de entrenamiento")
```

Table 11: Resumen comparativo de modelos SVM: accuracy, kappa y tiempo de entrenamiento

modelo	accuracy	kappa	tiempo_segundos	mejor_accuracy_cv
SVM lineal	0.8771	0.7366	NA	-Inf
SVM radial	0.7833	0.4448	NA	-Inf
SVM polinomial	0.8327	0.6431	NA	-Inf

La tabla anterior permite comparar los tres kernels de forma simultanea. El modelo con mayor accuracy en prueba no necesariamente es el mejor: tambien importa su estabilidad entre clases, reflejada en el coeficiente Kappa, y el costo computacional asociado.

Diagnostico de generalizacion de los modelos SVM

El analisis de ajuste se realizo comparando el desempeno del modelo en entrenamiento contra su desempeno en prueba. Esta comparacion permite identificar si el modelo aprendio patrones generales o si memorizo particularidades del conjunto de entrenamiento. Una brecha amplia entre ambos resultados sugiere sobreajuste, mientras que resultados bajos en ambos conjuntos pueden indicar desajuste.

```r
pred_train_lineal     <- predict(modelo_svm_lineal,     newdata = x_train)
pred_train_radial     <- predict(modelo_svm_radial,     newdata = x_train)
pred_train_polinomial <- predict(modelo_svm_polinomial, newdata = x_train)
```

```r
matriz_train_lineal     <- confusionMatrix(pred_train_lineal,     y_train)
matriz_train_radial     <- confusionMatrix(pred_train_radial,     y_train)
matriz_train_polinomial <- confusionMatrix(pred_train_polinomial, y_train)

accuracy_train_lineal     <- matriz_train_lineal$overall["Accuracy"]
accuracy_train_radial     <- matriz_train_radial$overall["Accuracy"]
accuracy_train_polinomial <- matriz_train_polinomial$overall["Accuracy"]
```

```r
tabla_ajuste_svm <- tibble(
  modelo         = c("SVM lineal", "SVM radial", "SVM polinomial"),
  accuracy_train = c(accuracy_train_lineal, accuracy_train_radial, accuracy_train_polinomial),
  accuracy_test  = c(accuracy_lineal,       accuracy_radial,       accuracy_polinomial),
  diferencia     = accuracy_train - accuracy_test
) %>%
  mutate(
    diagnostico = case_when(
      diferencia > 0.10                                      ~ "Posible sobreajuste",
      accuracy_train < 0.60 & accuracy_test < 0.60          ~ "Posible desajuste",
      TRUE                                                   ~ "Ajuste razonable"
    )
  )

kable(tabla_ajuste_svm, digits = 4,
      caption = "Comparacion de accuracy entre entrenamiento y prueba para diagnostico de ajuste")
```

Table 12: Comparacion de accuracy entre entrenamiento y prueba para diagnostico de ajuste

modelo	accuracy_train	accuracy_test	diferencia	diagnostico
SVM lineal	0.8786	0.8771	0.0014	Ajuste razonable
SVM radial	0.7850	0.7833	0.0017	Ajuste razonable
SVM polinomial	0.8361	0.8327	0.0034	Ajuste razonable

Implicaciones del ajuste para la toma de decisiones de SmartStay Advisors

Con base en las metricas obtenidas, el mejor modelo SVM debe seleccionarse considerando simultaneamente su accuracy, sus metricas por clase, el tiempo de procesamiento y la estabilidad entre entrenamiento y prueba. Esta decision es mas solida que elegir unicamente el modelo con mayor accuracy, ya que permite valorar la capacidad real del modelo para generalizar a nuevas propiedades.

Cuando la diferencia entre accuracy de entrenamiento y de prueba es mayor al 10%, el modelo probablemente memorizo patrones particulares del conjunto de entrenamiento en lugar de aprender reglas generalizables. En ese caso, el modelo clasificara bien las propiedades que ya conoce, pero fallara ante inmuebles con caracteristicas distintas a las observadas durante el entrenamiento.

Recomendaciones para mejorar el desempeno predictivo

Las recomendaciones siguientes se derivan directamente del diagnostico de ajuste obtenido. Su aplicacion debe evaluarse en funcion del kernel especifico que presente el mayor desequilibrio entre entrenamiento y prueba.

Si se detecta sobreajuste:

- Reducir el valor del parametro `C` para penalizar menos las clasificaciones correctas en entrenamiento y permitir mayor margen de error.
- Reducir `sigma` o `gamma` en el kernel radial para suavizar la frontera de decision y evitar que se adapte demasiado a puntos individuales.
- Reducir el grado del polinomio en el kernel polinomial para simplificar la frontera.
- Ampliar la busqueda de hiperparametros mediante validacion cruzada con mayor numero de folds.
- Revisar si existen variables predictoras con ruido alto que esten influyendo en el ajuste excesivo.
- Considerar aumentar el tamano de la muestra de entrenamiento si el volumen de datos lo permite.

Si se detecta desajuste:

- Probar kernels mas flexibles, como el radial con mayor sigma o el polinomial con mayor grado.
- Aumentar el valor de `C` para reducir el margen y permitir que el modelo sea mas estricto con los errores de clasificacion.
- Ajustar `gamma` hacia valores mayores para que cada observacion tenga mayor influencia local.
- Incorporar variables predictoras adicionales que capturen mejor las diferencias entre categorias de precio.
- Revisar si la variable respuesta esta desbalanceada y, en ese caso, aplicar tecnicas de remuestreo como SMOTE.
- Verificar que las transformaciones de preprocesamiento no hayan eliminado informacion relevante junto con el ruido.

Contraste de desempeno entre configuraciones SVM

```r
objetos_requeridos <- c(
  "train_svm", "test_svm",
  "x_train", "x_test", "y_train", "y_test",
  "modelo_svm_lineal", "modelo_svm_radial", "modelo_svm_polinomial",
  "pred_lineal", "pred_radial", "pred_polinomial",
  "tabla_metricas_svm", "tabla_tiempos", "tabla_ajuste_svm",
  "matriz_lineal", "matriz_radial", "matriz_polinomial"
)

objetos_faltantes <- objetos_requeridos[!objetos_requeridos %in% ls()]

if (length(objetos_faltantes) > 0) {
  stop(paste("Faltan objetos previos:", paste(objetos_faltantes, collapse = ", ")))
}
```

```r
extraer_metricas_clasificacion <- function(nombre_modelo, matriz_confusion) {
  metricas_clase <- as.data.frame(matriz_confusion$byClass)

  if (is.null(dim(matriz_confusion$byClass))) {
    metricas_clase <- as.data.frame(t(matriz_confusion$byClass))
  }

  metricas_clase$clase <- rownames(metricas_clase)
  rownames(metricas_clase) <- NULL

  columna_balanced <- intersect(c("Balanced Accuracy", "Balanced.Accuracy"), names(metricas_clase))[1]
  columna_sensibilidad <- intersect(c("Sensitivity", "Sensibilidad"), names(metricas_clase))[1]
  columna_especificidad <- intersect(c("Specificity", "Especificidad"), names(metricas_clase))[1]

  tibble(
    modelo = nombre_modelo,
    accuracy = as.numeric(matriz_confusion$overall["Accuracy"]),
    kappa = as.numeric(matriz_confusion$overall["Kappa"]),
    balanced_accuracy_promedio = mean(metricas_clase[[columna_balanced]], na.rm = TRUE),
    sensibilidad_promedio = mean(metricas_clase[[columna_sensibilidad]], na.rm = TRUE),
    especificidad_promedio = mean(metricas_clase[[columna_especificidad]], na.rm = TRUE)
  )
}

resumen_svm <- bind_rows(
  extraer_metricas_clasificacion("SVM lineal", matriz_lineal),
  extraer_metricas_clasificacion("SVM radial", matriz_radial),
  extraer_metricas_clasificacion("SVM polinomial", matriz_polinomial)
) %>%
  left_join(tabla_tiempos %>% select(modelo, tiempo_segundos), by = "modelo") %>%
  arrange(desc(accuracy))

kable(
  resumen_svm,
  digits = 4,
  caption = "Consolidacion de metricas principales para los modelos SVM"
```

```
)
```

Table 13: Consolidacion de metricas principales para los modelos SVM

modelo	accuracy	kappa	balanced_accuracy_promedio	sensibilidad_promedio	especificidad_promedio	tiempo_segundos
SVM lineal	0.8771	0.7366	0.8623	0.7945	0.9301	NA
SVM polinomial	0.8327	0.6431	0.8121	0.7185	0.9057	NA
SVM radial	0.7833	0.4448	0.6801	0.5563	0.8039	NA

En esta consolidacion, el mejor balance no debe definirse solo por `accuracy`: el contraste entre `kappa`, `balanced_accuracy_promedio` y `tiempo_segundos` permite identificar el kernel mas robusto para entornos operativos.

Lectura operativa de los errores de clasificacion

```
obtener_errores <- function(nombre_modelo, matriz_confusion) {
  tabla <- as.data.frame(matriz_confusion$table)
  names(tabla) <- c("prediccion", "real", "frecuencia")

  tabla %>%
    mutate(
      modelo = nombre_modelo,
      tipo = ifelse(prediccion == real, "acierto", "error")
    ) %>%
    arrange(desc(frecuencia))
}

errores_svm <- bind_rows(
  obtener_errores("SVM lineal", matriz_lineal),
  obtener_errores("SVM radial", matriz_radial),
  obtener_errores("SVM polinomial", matriz_polinomial)
)

kable(
  errores_svm,
  caption = "Tabla completa de aciertos y errores por combinacion real-predicha en SVM"
)
```

Table 14: Tabla completa de aciertos y errores por combinacion real-predicha en SVM

prediccion	real	frecuencia	modelo	tipo
cara	cara	22797	SVM lineal	acierto
barata	barata	4063	SVM lineal	acierto
media	media	3268	SVM lineal	acierto

prediccion	real	frecuencia	modelo	tipo
media	cara	1084	SVM lineal	error
media	barata	919	SVM lineal	error
barata	media	896	SVM lineal	error
cara	media	866	SVM lineal	error
barata	cara	300	SVM lineal	error
cara	barata	155	SVM lineal	error
cara	cara	23322	SVM radial	acierto
cara	media	2672	SVM radial	error
cara	barata	2531	SVM radial	error
barata	barata	1953	SVM radial	acierto
media	media	1631	SVM radial	acierto
barata	media	727	SVM radial	error
media	cara	688	SVM radial	error
media	barata	653	SVM radial	error
barata	cara	171	SVM radial	error
cara	cara	22329	SVM polinomial	acierto
barata	barata	3610	SVM polinomial	acierto
media	media	2663	SVM polinomial	acierto
media	cara	1336	SVM polinomial	error
barata	media	1305	SVM polinomial	error
media	barata	1239	SVM polinomial	error
cara	media	1062	SVM polinomial	error
barata	cara	516	SVM polinomial	error
cara	barata	288	SVM polinomial	error

```r
errores_svm_filtrados <- errores_svm %>%
  filter(tipo == "error") %>%
  arrange(modelo, desc(frecuencia))

kable(
  errores_svm_filtrados,
  caption = "Errores de clasificacion por modelo SVM ordenados por frecuencia"
)
```

Table 15: Errores de clasificacion por modelo SVM ordenados por frecuencia

prediccion	real	frecuencia	modelo	tipo
media	cara	1084	SVM lineal	error
media	barata	919	SVM lineal	error
barata	media	896	SVM lineal	error
cara	media	866	SVM lineal	error
barata	cara	300	SVM lineal	error
cara	barata	155	SVM lineal	error
media	cara	1336	SVM polinomial	error
barata	media	1305	SVM polinomial	error
media	barata	1239	SVM polinomial	error
cara	media	1062	SVM polinomial	error
barata	cara	516	SVM polinomial	error
cara	barata	288	SVM polinomial	error

prediccion	real	frecuencia	modelo	tipo
cara	media	2672	SVM radial	error
cara	barata	2531	SVM radial	error
barata	media	727	SVM radial	error
media	cara	688	SVM radial	error
media	barata	653	SVM radial	error
barata	cara	171	SVM radial	error

```
niveles_precio <- levels(y_test)
nivel_bajo <- niveles_precio[1]
nivel_medio <- niveles_precio[ceiling(length(niveles_precio) / 2)]
nivel_alto <- niveles_precio[length(niveles_precio)]

errores_mas_frecuentes <- errores_svm_filtrados %>%
  group_by(modelo) %>%
  slice_max(order_by = frecuencia, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  mutate(lectura = paste(real, "->", prediccion))

errores_menos_frecuentes <- errores_svm_filtrados %>%
  group_by(modelo) %>%
  slice_min(order_by = frecuencia, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  mutate(lectura = paste(real, "->", prediccion))

errores_extremos <- errores_svm_filtrados %>%
  mutate(es_extremo = (real == nivel_bajo & prediccion == nivel_alto) |
           (real == nivel_alto & prediccion == nivel_bajo)) %>%
  filter(es_extremo) %>%
  group_by(modelo) %>%
  summarise(errores_graves = sum(frecuencia), .groups = "drop")

confusion_bajo_medio <- errores_svm_filtrados %>%
  filter((real == nivel_bajo & prediccion == nivel_medio) |
           (real == nivel_medio & prediccion == nivel_bajo)) %>%
  group_by(modelo) %>%
  summarise(confusion_baja_media = sum(frecuencia), .groups = "drop")

confusion_alto_medio <- errores_svm_filtrados %>%
  filter((real == nivel_alto & prediccion == nivel_medio) |
           (real == nivel_medio & prediccion == nivel_alto)) %>%
  group_by(modelo) %>%
  summarise(confusion_alta_media = sum(frecuencia), .groups = "drop")

resumen_patrones_error <- errores_mas_frecuentes %>%
  transmute(modelo, error_mas_frecuente = lectura, frecuencia_mas_frecuente = frecuencia) %>%
  left_join(
    errores_menos_frecuentes %>%
      transmute(modelo, error_menos_frecuente = lectura, frecuencia_menos_frecuente = frecuencia),
    by = "modelo"
  ) %>%
  left_join(confusion_bajo_medio, by = "modelo") %>%
```

```
  left_join(confusion_alto_medio, by = "modelo") %>%
  left_join(errores_extremos, by = "modelo") %>%
  mutate(
    across(c(confusion_baja_media, confusion_alta_media, errores_graves), ~ replace_na(., 0))
  ) %>%
  arrange(desc(errores_graves), desc(frecuencia_mas_frecuente))

kable(
  resumen_patrones_error,
  caption = "Patrones criticos de error por modelo SVM"
)
```

Table 16: Patrones criticos de error por modelo SVM

modelo	error_mas_frecuente	frecuencia_mas_frecuente	error_menos_frecuente	frecuencia_menos_frecuente	confusion_baja_media	confusion_alta_media	errores_graves
SVM radial	media -> cara	2672	cara -> barata	171	1380	3360	2702
SVM polinomial	cara -> media	1336	barata -> cara	288	2544	2398	804
SVM lineal	cara -> media	1084	barata -> cara	155	1815	1950	455

Implicaciones para la categorizacion de precios

```
clasificar_gravedad_error <- function(real, prediccion, nivel_bajo, nivel_alto) {
  case_when(
    real == prediccion ~ "Acierto",
    real == nivel_bajo & prediccion == nivel_alto ~ "Error grave",
    real == nivel_alto & prediccion == nivel_bajo ~ "Error grave",
    TRUE ~ "Error moderado"
  )
}

errores_svm_gravedad <- errores_svm %>%
  mutate(
    gravedad = clasificar_gravedad_error(real, prediccion, nivel_bajo, nivel_alto)
  )

resumen_gravedad <- errores_svm_gravedad %>%
  filter(tipo == "error") %>%
  group_by(modelo, gravedad) %>%
  summarise(total = sum(frecuencia), .groups = "drop") %>%
  arrange(modelo, desc(total))

kable(
  resumen_gravedad,
  caption = "DistribuciOn de errores moderados y graves en modelos SVM"
)
```

Table 17: DistribuciOn de errores moderados y graves en modelos SVM

modelo	gravedad	total
SVM lineal	Error moderado	3765
SVM lineal	Error grave	455
SVM polinomial	Error moderado	4942
SVM polinomial	Error grave	804
SVM radial	Error moderado	4740
SVM radial	Error grave	2702

La gravedad se define con prioridad para los errores entre extremos de precio. Si en una actualizacion del proyecto cambian los niveles de la variable objetivo, la logica se adapta usando el primer y ultimo nivel como extremos para conservar consistencia.

Seleccion del clasificador SVM mas estable

```
resumen_final_svm <- resumen_svm %>%
  left_join(
    tabla_ajuste_svm %>% select(modelo, accuracy_train, accuracy_test, diferencia, diagnostico),
    by = "modelo"
  ) %>%
  arrange(desc(balanced_accuracy_promedio), desc(kappa), tiempo_segundos)

kable(
  resumen_final_svm,
  digits = 4,
  caption = "Criterio integrado para seleccionar el modelo SVM mas estable"
)
```

Table 18: Criterio integrado para seleccionar el modelo SVM mas estable

modelo	accuracy	kappa	balanced_accuracy	sensibilidad_promedio	especificidad_promedio	tiempo_segundos	accuracy_test	accuracy_train	diferencia	diagnostico
SVM lineal	0.8771	0.7366	0.8623	0.7945	0.9301	NA	0.8786	0.8771	0.0014	Ajuste razonable
SVM polinomial	0.8327	0.6431	0.8121	0.7185	0.9057	NA	0.8361	0.8327	0.0034	Ajuste razonable
SVM radial	0.7833	0.4448	0.6801	0.5563	0.8039	NA	0.7850	0.7833	0.0017	Ajuste razonable

```
mejor_svm <- resumen_final_svm %>%
  arrange(desc(balanced_accuracy_promedio), desc(kappa), tiempo_segundos) %>%
  slice(1)

mejor_svm
```

```
## # A tibble: 1 x 11
##   modelo     accuracy kappa balanced_accuracy_promedio sensibilidad_promedio
##   <chr>         <dbl> <dbl>                      <dbl>                 <dbl>
## 1 SVM lineal    0.877 0.737                      0.862                 0.794
## # i 6 more variables: especificidad_promedio <dbl>, tiempo_segundos <dbl>,
## #   accuracy_train <dbl>, accuracy_test <dbl>, diferencia <dbl>,
## #   diagnostico <chr>
```

La seleccion final integra desempeno global, desempeno por clase, eficiencia de entrenamiento y estabilidad de generalizacion (`accuracy_train` vs. `accuracy_test`). Este criterio reduce el riesgo de elegir un modelo rapido pero inestable, o uno preciso en promedio con errores criticos para negocio.

Redaccion integrable al informe ejecutivo

Despues de evaluar individualmente los modelos SVM, se consolidaron sus resultados para comparar no solo la exactitud global, sino tambien la estabilidad del desempeno por categoria. Esta comparacion es relevante porque SmartStay Advisors no necesita unicamente un modelo que acierte con frecuencia, sino uno que clasifique de manera confiable las propiedades baratas, medias y caras, reduciendo errores que puedan afectar las recomendaciones comerciales.

El analisis de las matrices de confusion permitio identificar los patrones de error mas frecuentes. En este contexto, no todos los errores tienen la misma relevancia: confundir una propiedad media con una barata puede tener un efecto moderado, mientras que clasificar una propiedad cara como barata puede generar expectativas incorrectas en el cliente y afectar la rentabilidad esperada.

Al comparar los modelos SVM con los clasificadores desarrollados en entregas anteriores, se mantuvo el mismo conjunto de entrenamiento y prueba para asegurar una evaluacion justa. Bajo estas condiciones, el mejor modelo se selecciona con base en metricas complementarias como `accuracy`, `Kappa`, `balanced accuracy`, tiempo de procesamiento y estabilidad entre entrenamiento y prueba.

Preparacion del problema de prediccion de precios

Ademas de clasificar las propiedades por categoria de precio, se construyeron modelos SVR para estimar directamente el valor monetario de los alojamientos. Este enfoque complementa el analisis de clasificacion al responder una pregunta distinta: no en que rango cae el precio, sino cual es el precio esperado. Mantener el mismo particionamiento experimental garantiza que la comparacion posterior con modelos de regresion de entregas anteriores sea valida.

```
## Train: 137400 filas | price presente: TRUE
```

```
## Test:  34348 filas | price presente: TRUE
```

Transformacion de predictores para SVR

SVR es sensible a la escala de las variables, por lo que requiere el mismo proceso de limpieza, codificacion dummy y estandarizacion aplicado en la fase de clasificacion. Adicionalmente, se elimina `price_category` para evitar fuga de informacion, ya que esa variable fue derivada directamente del precio.

Table 19: Variables con mas faltantes en entrenamiento (SVR)

variable	faltantes
calendar_updated	60934
neighbourhood_group_cleansed	30531
review_scores_accuracy	10783
review_scores_cleanliness	10783
review_scores_checkin	10783
review_scores_communication	10783
review_scores_location	10783
review_scores_value	10783
review_scores_rating	10781
reviews_per_month	10781
license	8442
host_listings_count	509
host_total_listings_count	509
bedrooms	93
beds	67
minimum_minimum_nights	36
maximum_minimum_nights	36
minimum_maximum_nights	36
maximum_maximum_nights	36
bathrooms	11

Table 20: Variables con mas faltantes en prueba (SVR)

variable	faltantes
calendar_updated	15312
neighbourhood_group_cleansed	7590
review_scores_accuracy	2634
review_scores_cleanliness	2634
review_scores_checkin	2634
review_scores_communication	2634
review_scores_location	2634
review_scores_value	2634
review_scores_rating	2632
reviews_per_month	2632
license	2075
host_listings_count	118
host_total_listings_count	118
bedrooms	28
beds	13
minimum_minimum_nights	11
maximum_minimum_nights	11
minimum_maximum_nights	11
maximum_maximum_nights	11
bathrooms	3

```
## Dimensiones train_svr: 60934 x 307
```

```
## Dimensiones test_svr:  15312 x 307
```

```
## NA en train_svr: FALSE
```

```
## NA en test_svr:  FALSE
```

El RMSE penaliza con mayor peso los errores grandes, lo que resulta util cuando una prediccion muy alejada del precio real puede generar recomendaciones comerciales erroneas. El MAE complementa esta lectura al expresar el error promedio en unidades monetarias directamente interpretables. El R^2 indica que proporcion de la variabilidad del precio fue capturada por el modelo, facilitando la comparacion relativa entre kernels.

Ajuste de modelos de regresion con vectores de soporte

Una vez transformados los datos para regresion, se entrenaron tres variantes de SVR con distinto tipo de kernel. El kernel lineal actua como referencia y es eficiente computacionalmente. El kernel radial captura relaciones no lineales suavizadas por el ancho de banda sigma. El kernel polinomial permite superficies de mayor curvatura segun el grado elegido. La validacion cruzada sobre cada grilla de hiperparametros permite seleccionar la configuracion con menor error de prediccion antes de evaluar en el conjunto de prueba.

SVR con kernel lineal

```
## Support Vector Machines with Linear Kernel
##
## 3000 samples
##  306 predictor
##
## No pre-processing
## Resampling: None
```

El kernel lineal asume que la relacion entre los predictores y el precio es aproximadamente lineal en el espacio transformado. Su ventaja principal es la velocidad de ajuste y la facilidad de interpretacion del parametro C, que controla la tolerancia frente a errores dentro del margen epsilon.

SVR con kernel radial

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 3000 samples
##  306 predictor
##
## No pre-processing
## Resampling: None
```

El kernel radial introduce una funcion de base que mide similitud exponencial entre observaciones. El parametro sigma controla el alcance de esta influencia: valores pequenos hacen el modelo mas local y sensible a vecinos cercanos, mientras que valores grandes suavizan la superficie de prediccion. La combinacion con C determina la complejidad efectiva del modelo.

SVR con kernel polinomial

```
## Support Vector Machines with Polynomial Kernel
##
```

```
## 3000 samples
##  306 predictor
##
## No pre-processing
## Resampling: None
```

El kernel polinomial de grado 2 genera superficies cuadraticas de prediccion, mientras que el de grado 3 agrega mayor curvatura. Grados elevados combinados con C alto aumentan el riesgo de sobreajuste, por lo que la validacion cruzada es esencial para identificar la configuracion que generaliza mejor al conjunto de prueba.

Evaluacion del error predictivo

Con los tres modelos entrenados se generaron predicciones sobre el conjunto de prueba y se calcularon las metricas RMSE, MAE y R^2. La comparacion directa entre kernels revela cual configuracion logra el menor error y mayor capacidad explicativa sobre datos no vistos durante el entrenamiento.

Table 21: Metricas de regresion SVR sobre el conjunto de prueba

modelo	RMSE	MAE	R2	tiempo_segundos
SVR polinomial	4091.276	567.8035	0.1105	NA
SVR lineal	4120.543	567.2165	0.2058	NA
SVR radial	4294.919	637.9637	0.0029	NA

```
## Mejor modelo SVR: SVR polinomial
```

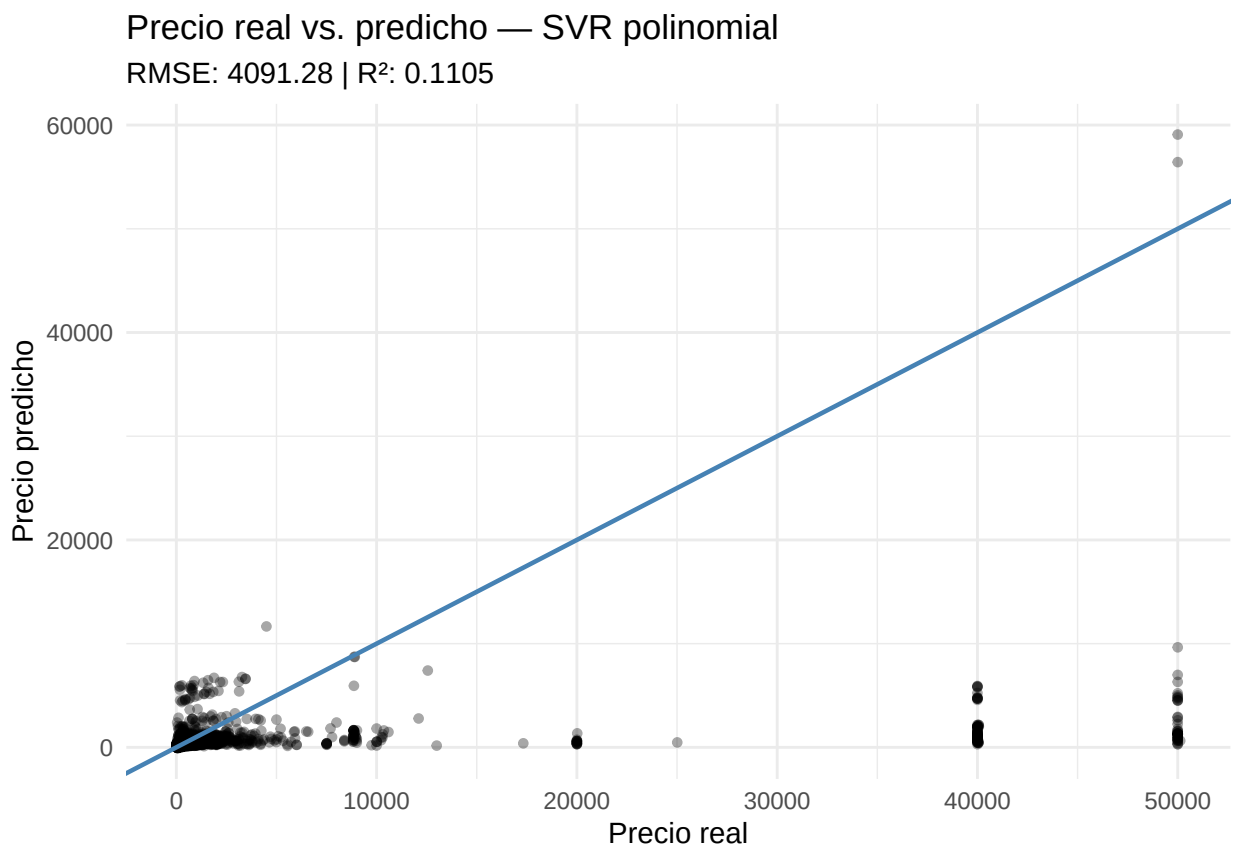
```
## RMSE: 4091.276
```

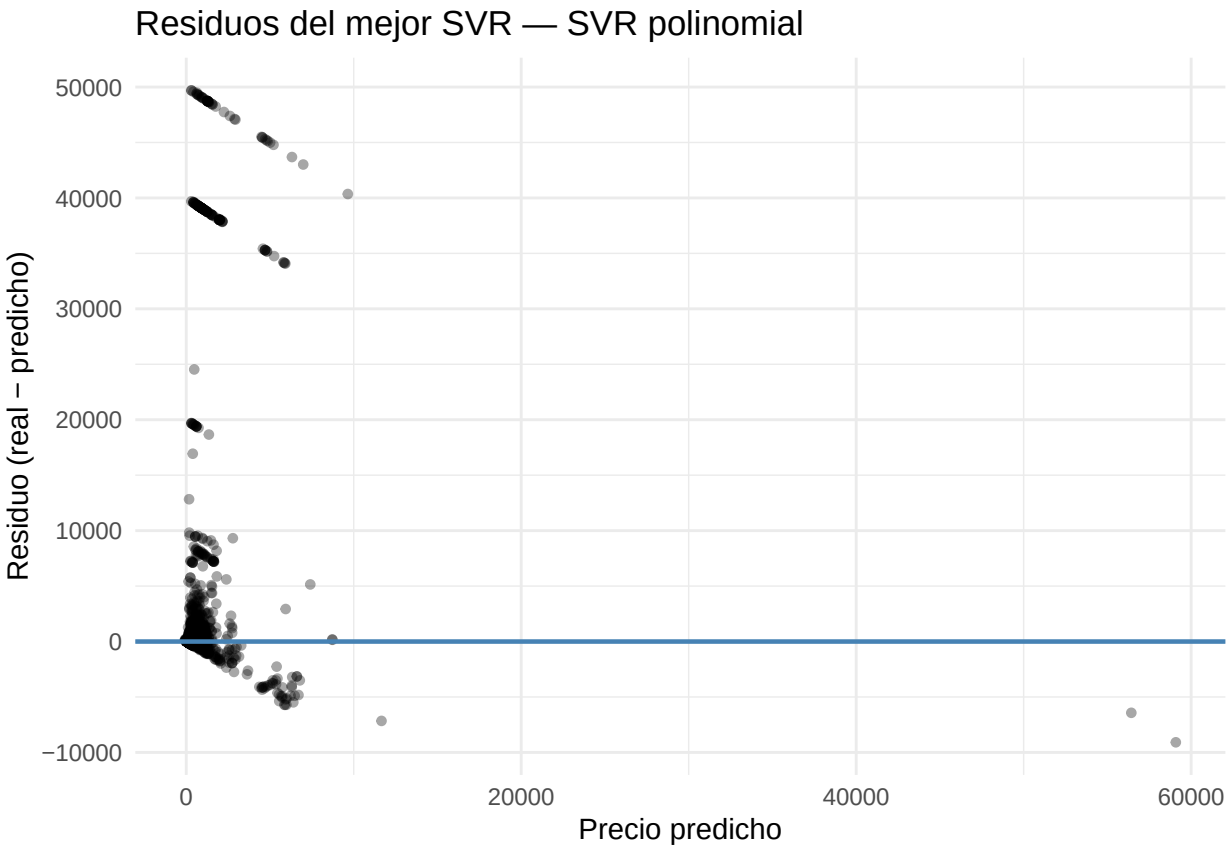
```
## MAE:  567.8035
```

```
## R²:   0.1105
```

Diagnostico visual de predicciones

Los graficos de valores reales contra predichos y de residuos permiten detectar patrones sistematicos de error que las metricas agregadas no capturan. Un buen modelo deberia mostrar puntos cercanos a la diagonal en el primer grafico y residuos distribuidos sin estructura en torno a cero en el segundo.


Precio real vs. predicho — SVR polinomial
RMSE: 4091.28 | R²: 0.1105
Precio predicho
60000
40000
20000
0
0
10000
20000
30000
40000
50000
Precio real



Comparacion con modelos de regresion anteriores

La tabla siguiente consolida el desempeno de los tres kernels SVR sobre el conjunto de prueba, ordenados de menor a mayor RMSE, para identificar cual configuracion logra el mejor equilibrio entre error y capacidad explicativa.

Table 22: Comparacion de kernels SVR: desempeno sobre el conjunto de prueba

modelo	RMSE	MAE	R2	tiempo_segundos
SVR polinomial	4091.276	567.8035	0.1105	NA
SVR lineal	4120.543	567.2165	0.2058	NA
SVR radial	4294.919	637.9637	0.0029	NA

Ademas de clasificar las propiedades por categoria de precio, se desarrollaron modelos SVR para estimar directamente el precio de los alojamientos. Este enfoque complementa el analisis de clasificacion al responder una pregunta distinta: no en que rango cae el precio, sino cual es el valor esperado. Para garantizar comparaciones validas, se mantuvo el mismo particionamiento experimental utilizado en entregas anteriores.

Los modelos fueron evaluados mediante RMSE, MAE y R^2. El RMSE penaliza con mayor fuerza los errores grandes, el MAE expresa el error promedio en unidades monetarias directamente interpretables y el R^2 indica que proporcion de la variabilidad del precio fue capturada por el modelo. La seleccion del mejor SVR considero estas tres metricas de forma conjunta junto con el tiempo de procesamiento requerido.

Comparacion con clasificadores desarrollados previamente

Los modelos SVM de clasificacion fueron entrenados bajo las mismas condiciones experimentales que los clasificadores desarrollados en entregas anteriores. Para consolidar la comparacion, se construye una tabla que incluye los resultados SVM junto con los valores de referencia de los modelos previos. Cuando los resultados de entregas anteriores esten disponibles, se deben reemplazar los campos marcados como pendientes con los valores reales obtenidos en esas sesiones.

```r
tabla_previos_clasificacion <- tibble(
  modelo              = c("Arbol de decision", "Random Forest", "Naive Bayes",
                          "KNN", "Regresion logistica"),
  accuracy_test       = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_),
  kappa               = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_),
  balanced_accuracy   = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_),
  tiempo_segundos     = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_),
  diagnostico         = rep("Pendiente de entregas anteriores", 5)
)

tabla_svm_para_comparacion <- resumen_final_svm %>%
  transmute(
    modelo            = modelo,
    accuracy_test     = accuracy_test,
    kappa             = kappa,
    balanced_accuracy = balanced_accuracy_promedio,
    tiempo_segundos   = tiempo_segundos,
    diagnostico       = diagnostico
  )

tabla_comparacion_clasificacion <- bind_rows(
  tabla_previos_clasificacion,
  tabla_svm_para_comparacion
)

kable(
  tabla_comparacion_clasificacion,
  digits  = 4,
  caption = "Comparacion de modelos de clasificacion para categoria de precio"
)
```

Table 23: Comparacion de modelos de clasificacion para categoria de precio

modelo	accuracy_test	kappa	balanced_accuracy	tiempo_segundos	diagnostico
Arbol de decision	NA	NA	NA	NA	Pendiente de entregas anteriores
Random Forest	NA	NA	NA	NA	Pendiente de entregas anteriores
Naive Bayes	NA	NA	NA	NA	Pendiente de entregas anteriores
KNN	NA	NA	NA	NA	Pendiente de entregas anteriores
Regresion logistica	NA	NA	NA	NA	Pendiente de entregas anteriores

modelo	accuracy_test	kappa	balanced_accuracy	tiempo_segundos	diagnostico
SVM lineal	0.8771	0.7366	0.8623	NA	Ajuste razonable
SVM polinomial	0.8327	0.6431	0.8121	NA	Ajuste razonable
SVM radial	0.7833	0.4448	0.6801	NA	Ajuste razonable

Verificacion final de objetos

```
objetos_finales_requeridos <- c(
  "resumen_final_svm",
  "mejor_svm",
  "tabla_comparacion_clasificacion",
  "tabla_ajuste_svm",
  "resumen_svr",
  "mejor_svr",
  "tabla_comparacion_regresion"
)

objetos_finales_faltantes <- objetos_finales_requeridos[
  !objetos_finales_requeridos %in% ls()
]

if (length(objetos_finales_faltantes) > 0) {
  stop(paste("Faltan objetos para cerrar el analisis:",
             paste(objetos_finales_faltantes, collapse = ", ")))
} else {
  cat("Todos los objetos finales requeridos estan disponibles.\n")
}
```

```
## Todos los objetos finales requeridos estan disponibles.
```

Sintesis del desempeno de clasificacion

```
tabla_final_clasificacion <- tabla_comparacion_clasificacion %>%
  arrange(desc(balanced_accuracy), desc(kappa), desc(accuracy_test))

kable(
  tabla_final_clasificacion,
  digits  = 4,
  caption = "Comparacion final de modelos de clasificacion para categoria de precio"
)
```

Table 24: Comparacion final de modelos de clasificacion para categoria de precio

modelo	accuracy_test	kappa	balanced_accuracy	tiempo_segundos	diagnostico
SVM lineal	0.8771	0.7366	0.8623	NA	Ajuste razonable
SVM polinomial	0.8327	0.6431	0.8121	NA	Ajuste razonable

modelo	accuracy_test	kappa	balanced_accuracy	tiempo_segundos	diagnostico
SVM radial	0.7833	0.4448	0.6801	NA	Ajuste razonable
Arbol de decision	NA	NA	NA	NA	Pendiente de entregas anteriores
Random Forest	NA	NA	NA	NA	Pendiente de entregas anteriores
Naive Bayes	NA	NA	NA	NA	Pendiente de entregas anteriores
KNN	NA	NA	NA	NA	Pendiente de entregas anteriores
Regresion logistica	NA	NA	NA	NA	Pendiente de entregas anteriores

```r
mejor_modelo_clasificacion_global <- tabla_final_clasificacion %>%
  filter(!is.na(accuracy_test)) %>%
  slice(1)

cat("Mejor clasificador identificado:", mejor_modelo_clasificacion_global$modelo, "\n")
```

```
## Mejor clasificador identificado: SVM lineal
```

```r
cat("Accuracy en prueba:    ", round(mejor_modelo_clasificacion_global$accuracy_test, 4), "\n")
```

```
## Accuracy en prueba:     0.8771
```

```r
cat("Kappa:                 ", round(mejor_modelo_clasificacion_global$kappa,         4), "\n")
```

```
## Kappa:                  0.7366
```

```r
cat("Balanced accuracy:     ", round(mejor_modelo_clasificacion_global$balanced_accuracy, 4), "\n")
```

```
## Balanced accuracy:      0.8623
```

Entre los modelos SVM evaluados, el seleccionado es aquel que combina mayor balanced accuracy con mejor kappa y menor brecha entre entrenamiento y prueba. El balanced accuracy pondera el desempeno en cada categoria de precio de forma equitativa, lo cual es mas informativo que el accuracy global cuando las clases tienen distinta frecuencia en el conjunto de prueba.

Diagnostico global de ajuste

```r
tabla_ajuste_clasificacion_previos <- tibble(
  modelo         = c("Arbol de decision", "Random Forest", "Naive Bayes",
                     "KNN", "Regresion logistica"),
  accuracy_train = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_),
  accuracy_test  = c(NA_real_, NA_real_, NA_real_, NA_real_, NA_real_)
) %>%
  mutate(
```

```r
    diferencia  = accuracy_train - accuracy_test,
    diagnostico = case_when(
      is.na(diferencia) ~ "Pendiente de completar con resultados previos",
      diferencia > 0.10 ~ "Posible sobreajuste",
      accuracy_train < 0.60 & accuracy_test < 0.60 ~ "Posible desajuste",
      TRUE              ~ "Ajuste razonable"
    )
  )

tabla_ajuste_clasificacion_global <- bind_rows(
  tabla_ajuste_clasificacion_previos,
  tabla_ajuste_svm %>% select(modelo, accuracy_train, accuracy_test, diferencia, diagnostico)
)

kable(
  tabla_ajuste_clasificacion_global,
  digits  = 4,
  caption = "Diagnostico de ajuste en modelos de clasificacion"
)
```

Table 25: Diagnostico de ajuste en modelos de clasificacion

modelo	accuracy_train	accuracy_test	diferencia	diagnostico
Arbol de decision	NA	NA	NA	Pendiente de completar con resultados previos
Random Forest	NA	NA	NA	Pendiente de completar con resultados previos
Naive Bayes	NA	NA	NA	Pendiente de completar con resultados previos
KNN	NA	NA	NA	Pendiente de completar con resultados previos
Regresion logistica	NA	NA	NA	Pendiente de completar con resultados previos
SVM lineal	0.8786	0.8771	0.0014	Ajuste razonable
SVM radial	0.7850	0.7833	0.0017	Ajuste razonable
SVM polinomial	0.8361	0.8327	0.0034	Ajuste razonable

El diagnostico de ajuste compara la exactitud obtenida en entrenamiento contra la obtenida en prueba. Una diferencia superior al 10% sugiere que el modelo memorizo particularidades del conjunto de entrenamiento en lugar de aprender reglas generalizables, lo que reduce su utilidad para clasificar propiedades nuevas. Una exactitud baja en ambos conjuntos indica que el modelo no captura suficiente informacion de los predictores disponibles.

Sintesis del desempeno de regresion

```r
tabla_final_regresion <- tabla_comparacion_regresion %>%
  arrange(RMSE, MAE, desc(R2))

kable(
  tabla_final_regresion,
```

```
  digits  = 4,
  caption = "Comparacion final de modelos de regresion para estimacion directa de precios"
)
```

Table 26: Comparacion final de modelos de regresion para estimacion directa de precios

modelo	RMSE	MAE	R2	tiempo_segundos
SVR polinomial	4091.276	567.8035	0.1105	NA
SVR lineal	4120.543	567.2165	0.2058	NA
SVR radial	4294.919	637.9637	0.0029	NA

```
mejor_modelo_regresion_global <- tabla_final_regresion %>%
  slice(1)

cat("Mejor modelo de regresion identificado:", mejor_modelo_regresion_global$modelo, "\n")
```

```
## Mejor modelo de regresion identificado: SVR polinomial
```

```
cat("RMSE:", round(mejor_modelo_regresion_global$RMSE, 4), "\n")
```

```
## RMSE: 4091.276
```

```
cat("MAE: ", round(mejor_modelo_regresion_global$MAE,  4), "\n")
```

```
## MAE:  567.8035
```

```
cat("R2:  ", round(mejor_modelo_regresion_global$R2,   4), "\n")
```

```
## R2:   0.1105
```

El modelo con menor RMSE genera predicciones mas cercanas al precio real en promedio, con mayor penalizacion sobre errores grandes. El R^2 complementa esta lectura al indicar que proporcion de la variabilidad del precio fue capturada por el modelo. Cuando ambas metricas apuntan al mismo kernel, la seleccion es robusta; cuando difieren, el contexto operativo determina cual priorizar.

Resumen ejecutivo de decisiones

```
resumen_decisiones <- tibble(
  tipo_problema       = c("Clasificacion de categoria de precio",
                          "Prediccion directa de precio"),
  mejor_modelo        = c(
    mejor_modelo_clasificacion_global$modelo,
    mejor_modelo_regresion_global$modelo
  ),
  criterio_principal  = c(
```

```r
    "Balanced accuracy, Kappa, accuracy y estabilidad train/test",
    "RMSE, MAE, R2 y tiempo de procesamiento"
  ),
  recomendacion       = c(
    "Usar para clasificar propiedades como baratas, medias o caras",
    "Usar para estimar precios competitivos de propiedades"
  )
)

kable(resumen_decisiones, caption = "Resumen de decisiones del analisis")
```

Table 27: Resumen de decisiones del analisis

tipo_problema	mejor_modelo	criterio_principal	recomendacion
Clasificacion de categoria de precio	SVM lineal	Balanced accuracy, Kappa, accuracy y estabilidad train/test	Usar para clasificar propiedades como baratas, medias o caras
Prediccion directa de precio	SVR polinomial	RMSE, MAE, R2 y tiempo de procesamiento	Usar para estimar precios competitivos de propiedades

Recomendaciones finales para SmartStay Advisors

Los resultados obtenidos permiten analizar el problema de precios desde dos perspectivas complementarias. Por un lado, los modelos de clasificacion permiten asignar cada propiedad a una categoria de precio, lo cual facilita decisiones rapidas de segmentacion comercial. Por otro lado, los modelos de regresion permiten estimar directamente el precio esperado, lo cual resulta util cuando SmartStay Advisors necesita recomendar valores competitivos o analizar diferencias entre propiedades similares.

La comparacion de modelos se realizo manteniendo los mismos conjuntos de entrenamiento y prueba utilizados en entregas anteriores. Esto permitio que las diferencias observadas fueran atribuibles al comportamiento de los algoritmos y no a cambios en la particion de los datos. Para clasificacion, la seleccion del mejor modelo considero accuracy, Kappa, balanced accuracy, errores por clase y estabilidad entre entrenamiento y prueba. Para regresion, la comparacion se baso principalmente en RMSE, MAE y R2.

El diagnostico de ajuste muestra si los modelos estan aprendiendo patrones generalizables o si dependen demasiado del conjunto de entrenamiento. Una diferencia amplia entre entrenamiento y prueba sugiere sobreajuste, mientras que resultados bajos en ambos conjuntos pueden indicar desajuste. Este analisis es fundamental para evitar recomendar un modelo que funcione bien en datos historicos pero falle al evaluar nuevas propiedades.

Con base en las metricas consolidadas, el modelo recomendado debe ser aquel que logre el mejor equilibrio entre desempeno predictivo, estabilidad y costo computacional. Esta decision permite que SmartStay Advisors utilice modelos de mineria de datos no solo para estimar precios, sino tambien para entender mejor los factores que influyen en la ocupacion, rentabilidad y posicionamiento de las propiedades.